

Звіт про веб-клієнт проєкту Remote File Folder Client (спрощений аналог Google Drive)

Уточкіна Яна

Етап: веб-клієнт (React/TypeScript) – проєктування,
використання та тестування
30 вересня 2026 р.

Анотація

Цей звіт документує **веб-клієнт на React** проєкту **Remote File Folder Client** (frontend/src/) – спрощеного аналога Google Drive з веб- і десктоп-клієнтами, Go-бекендом, PostgreSQL та синхронізацією локальної теки. У звіті описано внутрішню структуру клієнта, розглянуто його прецеденти використання зі скріншотами та задокументовано покриття модульними тестами, включно з функцією синхронізації теки – браузерним портом рушія синхронізації десктоп-клієнта, побудованим на File System Access API.

Зміст

1 Вступ	2
2 Архітектура веб-клієнта	2
2.1 Внутрішня структура	2
2.2 Синхронізація	3
3 Прецеденти використання та скріншоти	3
3.1 Реєстрація та автентифікація	3
3.2 Перегляд файлів: атрибути, стовпці, сортування, фільтрація . .	4
3.3 Перегляд вмісту файлу	7
3.4 Вивантаження, завантаження, видалення	8
3.5 Синхронізація локальної теки	11

4 Модульне тестування	12
4.1 Налаштування тестів	12
4.2 Покриття за областями	13
5 Висновки	14

1. Вступ

Веб-клієнт – один із двох фронтендів (другий – десктоп-клієнт на Swift) до єдиного Go API та бази PostgreSQL, самостійно розгорнутих на домашньому Ubuntu-сервері за Cloudflare Tunnel. Він покриває кожен основний прецедент роботи з робочою областю – реєстрацію, автентифікацію, перегляд файлів із сортованими/фільтрованими атрибутами, перегляд вмісту .java/.png, вивантаження (включно з drag-and-drop), завантаження, видалення – а в браузері на основі Chromium ще й синхронізацію теки за тим самим серверним протоколом diff, який використовує десктоп-клієнт. На відміну від десктоп-клієнта, браузер не може безперервно спостерігати за локальною текою у фоновому режимі, тож синхронізація тут – це разова дія за запитом користувача, а не завжди активна служба.

2. Архітектура веб-клієнта

2.1 Внутрішня структура

Тека з вихідним кодом веб-клієнта, frontend/src/, організована так:

- **App.tsx**: основний компонент DriveView – TanStack Query для стану сервера (пагінований useInfiniteQuery для видимого списку файлів, окремий useQuery для повного переліку розширень, що живить випадний список фільтра), TanStack Table для сортування та видимості стовпців, TanStack Virtual для віконного рендерингу рядків, а також обробка вивантаження методом drag-and-drop;
- **columns.tsx**: визначення стовпців таблиці файлів (ім'я, створено, змінено, автор вивантаження, автор редагування, дії), включно з тим, які розширення отримують клікабельне посилання попереднього перегляду в клітинці імені;
- **auth/**: AuthContext.tsx (React Context і хук, що зберігає залогіненого користувача та JWT access/refresh токени, збережені в localStorage) і LoginPage.tsx (об'єднана форма входу/реєстрації);
- **api/**: client.ts (REST-клієнт на основі fetch з політикою “одне оновлення – одна повторна спроба” для протермінованого access-токена, та chunked-шлях вивантаження для великих файлів) і types.ts (DTO для формату обміну даними);
- **SettingsModal.tsx / FilePreviewModal.tsx**: модальні вікна видимості стовпців та попереднього перегляду вмісту файлу, що використовують спільне модальне оформлення застосунку;
- **sync/**: функція синхронізації теки – hash.ts, folderScanner.ts, storage.ts, reconcile.ts (алгоритм синхронізації та стан) і SyncContext.tsx/SyncModal.tsx/SyncConflictsSection.tsx (її React-стан та інтерфейс).

Веб-клієнт звертається до Go API-сервера через HTTPS за тим самим публічним хостнеймом Cloudflare Tunnel, що й десктоп-клієнт, і відображає

ті самі записи файлів, з якими працюють і десктоп-клієнт, і сервер.

2.2 Синхронізація

Синхронізація теки дзеркалить рушій синхронізації десктоп-клієнта, адаптований до того, що реально може браузер: фоновий спостерігач за текою немає, тож кнопка **Sync now** запускає один цикл узгодження за запитом. Клік на неї: сканує обрану теку через `FileSystemDirectoryHandle` з `File System Access API`, обчислює маніфест (ім'я, розмір, SHA-256 для кожного файлу, хешування через `crypto.subtle`), надсилає його разом із останньою синхронізованою версією на `POST /api/sync/diff` – той самий ендпоінт і протокол, який використовує десктоп-клієнт – і застосовує отримані дії `download/delete_local/conflict`, перш ніж вивантажити нові чи змінені локальні файли та видалити віддалені копії локально видалених. Курсор синхронізації та відомості по кожному файлу зберігаються між перезавантаженнями сторінки в `IndexedDB`, поруч із дескриптором обраної теки (який браузер може повторно використати без повторного вибору теки користувачем, за умови свіжого дозволу на кожну сесію).

Дія `conflict` означає, що той самий файл змінився з обох боків від часу останньої синхронізації. Вона ніколи не розв'язується автоматично: користувачу показується локальний і віддалений розмір/редактор, а розв'язати конфлікт можна кнопками **Keep Local** (повторно вивантажити, перезаписавши віддалену копію), **Keep Remote** (перезаписати локальний файл) або **Keep Both** (зберегти локальну редакцію під перейменованою копією й прийняти віддалену версію) – точно відповідно до трибічного розв'язання десктоп-клієнта.

`File System Access API` доступний лише в браузерах на основі Chromium (Chrome, Edge, Brave); у будь-якому іншому браузері панель синхронізації показує просте повідомлення про це, а не намагається використати відсутній API.

3. Прецеденти використання та скріншоти

У цьому розділі розглянуто прецеденти використання веб-клієнта, кожен зі скріншотом працюючого застосунку.

3.1 Реєстрація та автентифікація

Реєстрація (рис. 1) та *вхід* (рис. 2) – це два режими однієї форми (`LoginPage.tsx`); успішний виклик будь-якого з них зберігає отриману пару JWT `access/refresh` та ім'я користувача в `localStorage` через `AuthContext`, після чого стають доступними всі інші подання.

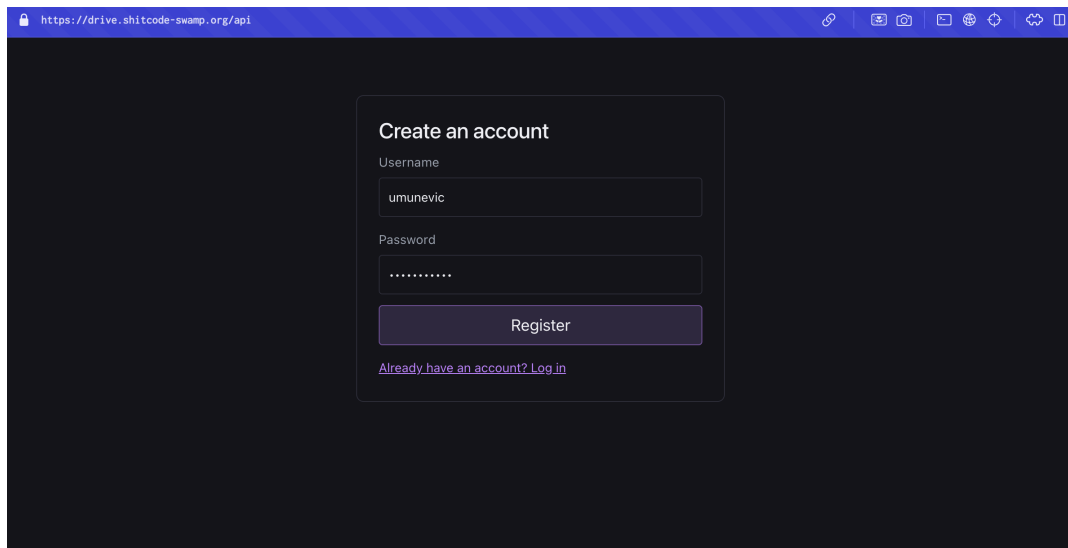


Рис. 1: Екран реєстрації

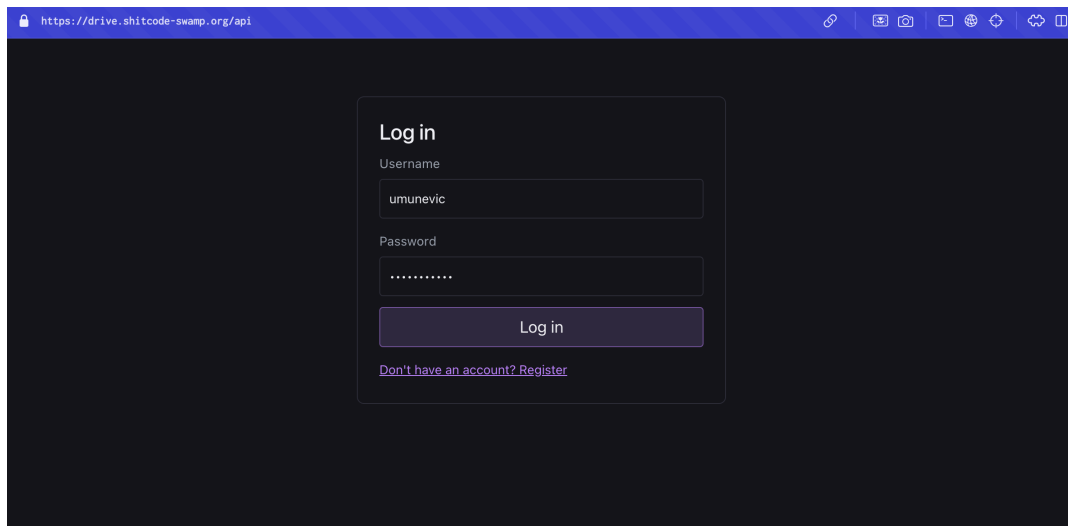


Рис. 2: Екран входу

3.2 Перегляд файлів: атрибути, стовпці, сортування, фільтрація

Рис. 3 показує основну таблицю файлів з усіма видимими стовпцями атрибутів (ім'я, створено, змінено, автор вивантаження, автор редагування); стовпці імені та дій приховати не можна, тоді як інші перемикаються через модальне вікно Settings (рис. 4).

Name ↑	Created ↓	Modified ↓	Uploaded by ↓	Edited by ↓	Actions
concrete-texture-black-wallpaper-background-photo.jpg	30/09/2026, 16:39	30/09/2026, 16:39	umunevic	umunevic	Download Delete
Funny Cartoon Car Crash Sound Effect-(117k).mp3	30/09/2026, 17:15	30/09/2026, 17:15	umunevic	umunevic	Download Delete
kursova_mi_22_23_6_2_(1).pdf	30/09/2026, 15:44	30/09/2026, 15:44	umunevic	umunevic	Download Delete
pngtree-fruit-fresh-tomato-png-image_9959799.png	30/09/2026, 16:39	30/09/2026, 16:39	umunevic	umunevic	Download Delete
Screenshot 2026-09-29 at 17.46.37.png	30/09/2026, 19:05	30/09/2026, 19:05	umunevic	umunevic	Download Delete
tg_image_4072793523.png	30/09/2026, 15:45	30/09/2026, 15:45	umunevic	umunevic	Download Delete
white-cloud-blue-sky-sea_74190-4488.avif	30/09/2026, 16:39	30/09/2026, 16:39	umunevic	umunevic	Download Delete
БШНФОП18ПЛ19_ДВС_Прикладні_алг_2025_2026.doc	30/09/2026, 17:15	30/09/2026, 17:15	umunevic	umunevic	Download Delete
Лекція_3.docx	30/09/2026, 15:43	30/09/2026, 15:43	umunevic	umunevic	Download Delete
НП ШІ 2025-26-1.pdf	30/09/2026, 16:39	30/09/2026, 16:39	umunevic	umunevic	Download Delete

Рис. 3: Основний список файлів з усіма видимими стовпцями атрибутів

Name ↑	Created ↓	Edited by ↓	Actions
concrete-texture-black-wallpaper-background-photo.jpg	30/09/2026, 16:39	umunevic	Download Delete
Funny Cartoon Car Crash Sound Effect-(117k).mp3	30/09/2026, 17:15	umunevic	Download Delete
kursova_mi_22_23_6_2_(1).pdf	30/09/2026, 15:44	umunevic	Download Delete
pngtree-fruit-f			Delete
Screenshot 20			Delete
tg_image_407			Delete
white-cloud-b			Delete
БШНФОП18П			Delete
Лекція_3.docx	30/09/2026, 15:43	umunevic	Download Delete
НП ШІ 2025-26-1.pdf	30/09/2026, 16:39	umunevic	Download Delete

Рис. 4: Модальне вікно Settings: перемикання стовпця, відмінного від імені

Сортування: клік на заголовок стовпця сортує таблицю за цим стовпцем, за зростанням або спаданням, що показується стрілкою біля заголовка (рис. 5 – сортування за “Edited by”). Сортування й фільтрація застосовуються на сервері – кожна зміна повторно запитує поточну сторінку в API з

відповідними параметрами запиту `sort/order/extension`, а не перевпорядковує вже завантажену сторінку в браузері.

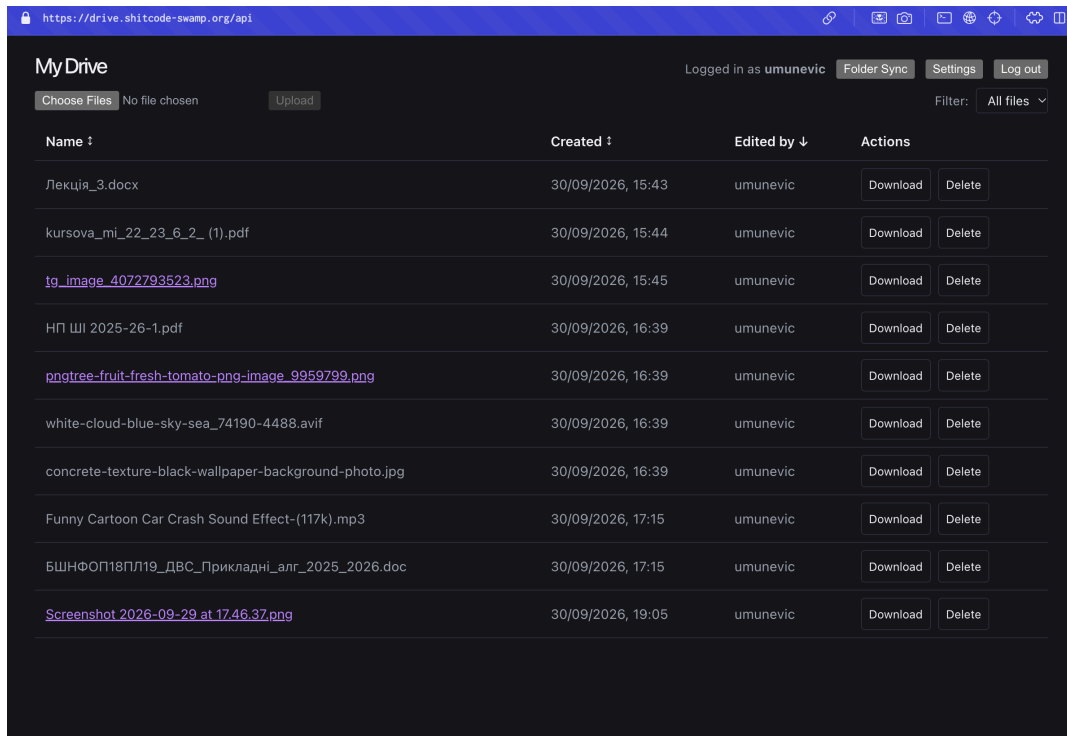


Рис. 5: Список файлів, відсортований за “Edited by”

Фільтрація: випадний список розширень, що формується з усіх розширень, реально присутніх серед файлів користувача (завантажується незалежно від поточного фільтра, тож перелік варіантів ніколи не звужується лише до обраного розширення), фільтрує видиму таблицю до лише цього розширення або назад до “All files” (рис. 6).

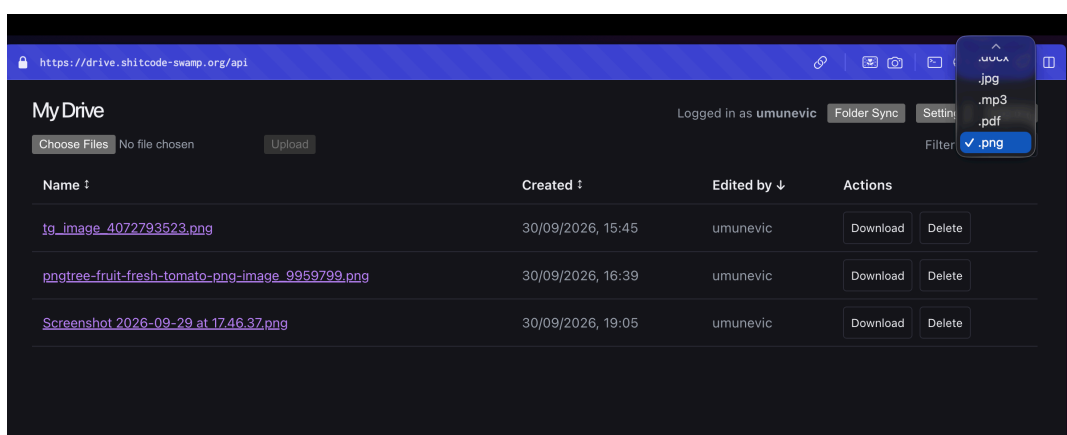


Рис. 6: Застосований фільтр за розширенням

3.3 Перегляд вмісту файлу

Клік на ім'я файлу показує його вміст, якщо розширення – .java (відображається як текст, рис. 7) або .png (відображається як зображення, рис. 8); ім'я файлу з будь-яким іншим розширенням – звичайний, неклікабельний текст: файл усе одно можна завантажити, просто не переглянути прямо в застосунку.

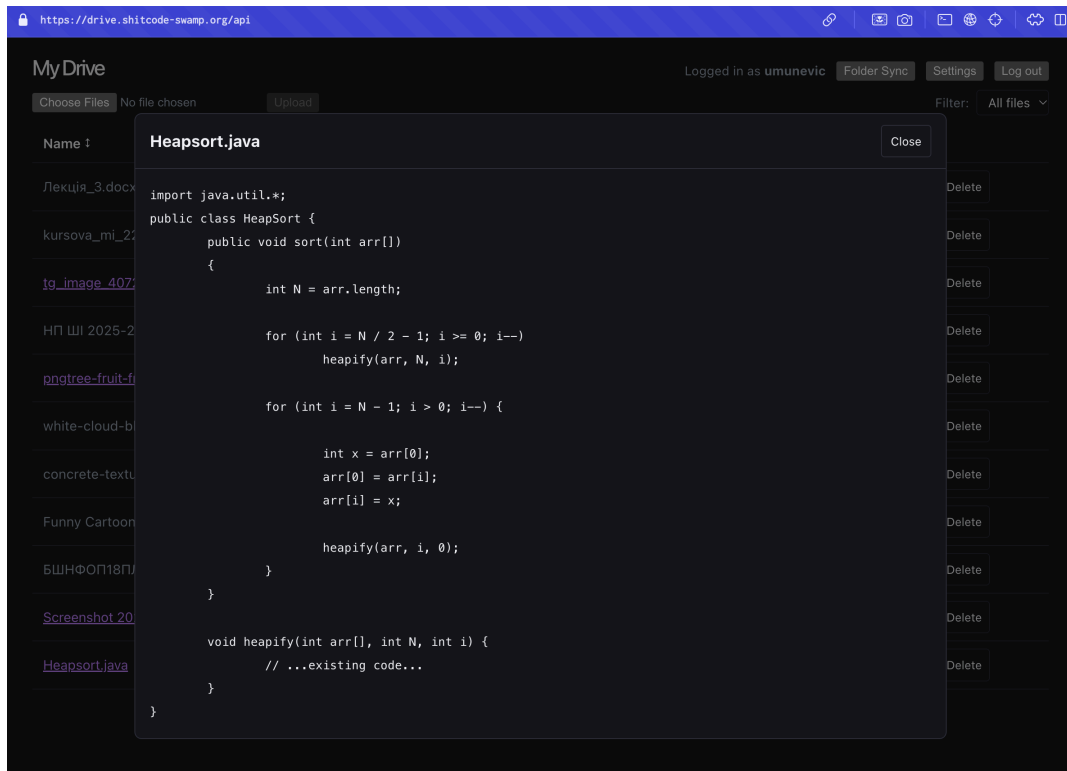


Рис. 7: Файл .java, показаний як текст

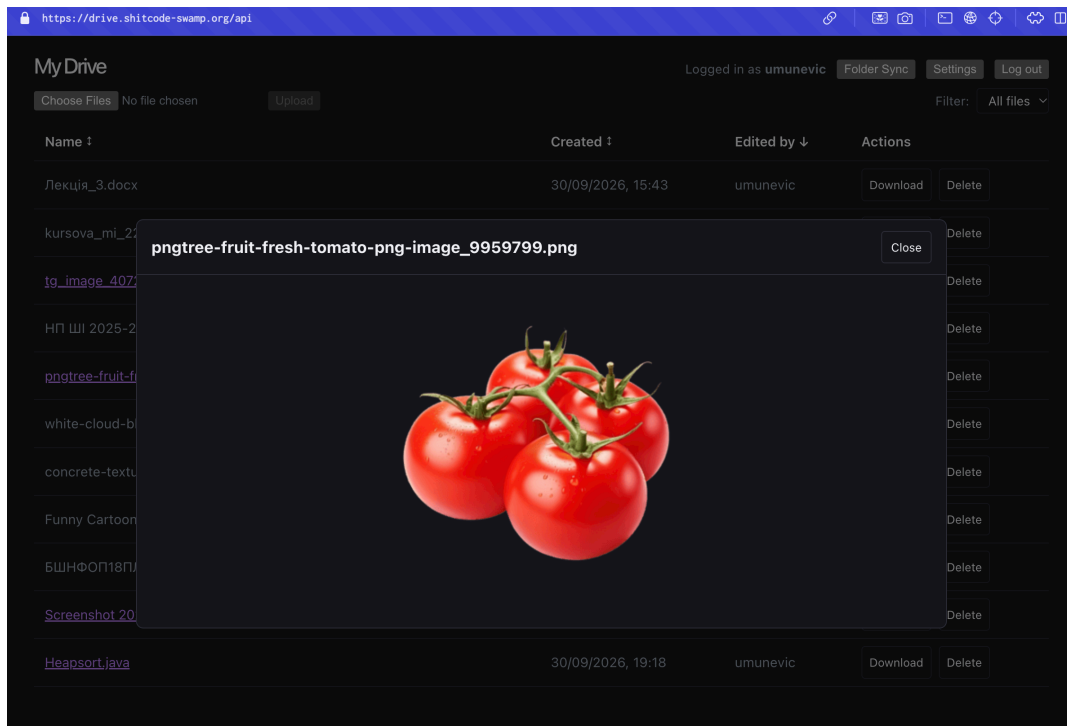


Рис. 8: Файл .png, показаний як зображення

3.4 Вивантаження, завантаження, видалення

Файли можна вивантажити або через вибір файлу і кнопку “Upload”, або перетягнувши їх у будь-яке місце сторінки – під час перетягування файлів з’являється накладка на всю сторінку (рис. 9). Рис. 10 показує вивантаження в процесі: великі файли розбиваються на chunk’и, щоб жоден окремий HTTP-запит не перевищував ліміт розміру тіла безкоштовного тарифу Cloudflare Tunnel, а індикатор прогресу для кожного файлу відображає кількість надісланих байтів по кожному chunk’у. Рис. 11 та 12 показують завантаження файлу та видалення файлу з робочої області.

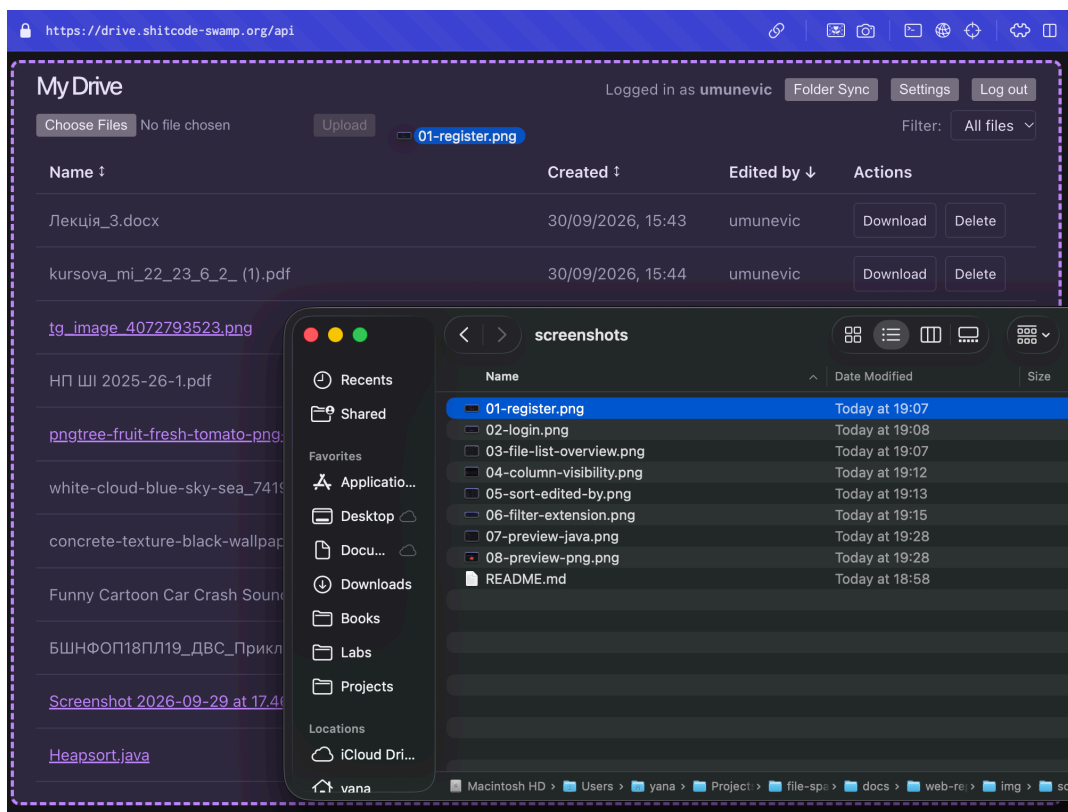


Рис. 9: Накладка перетягування, що з'являється під час перетягування файлів на сторінку

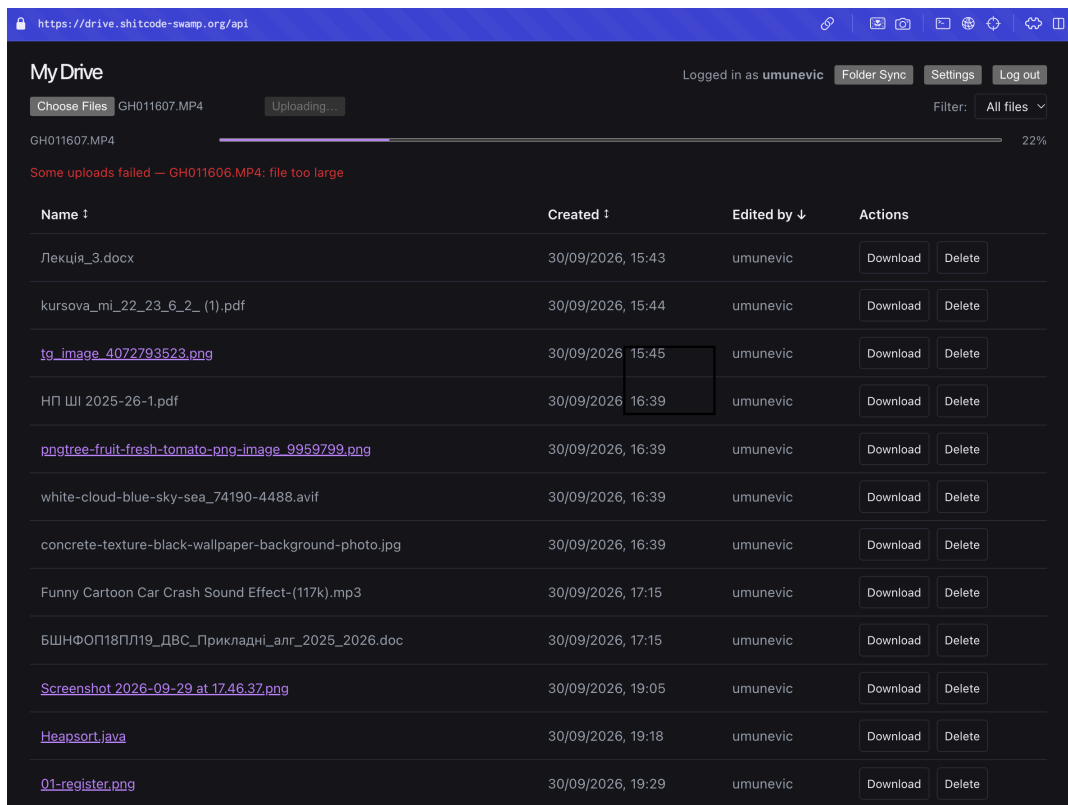


Рис. 10: Вивантаження файлу в процесі

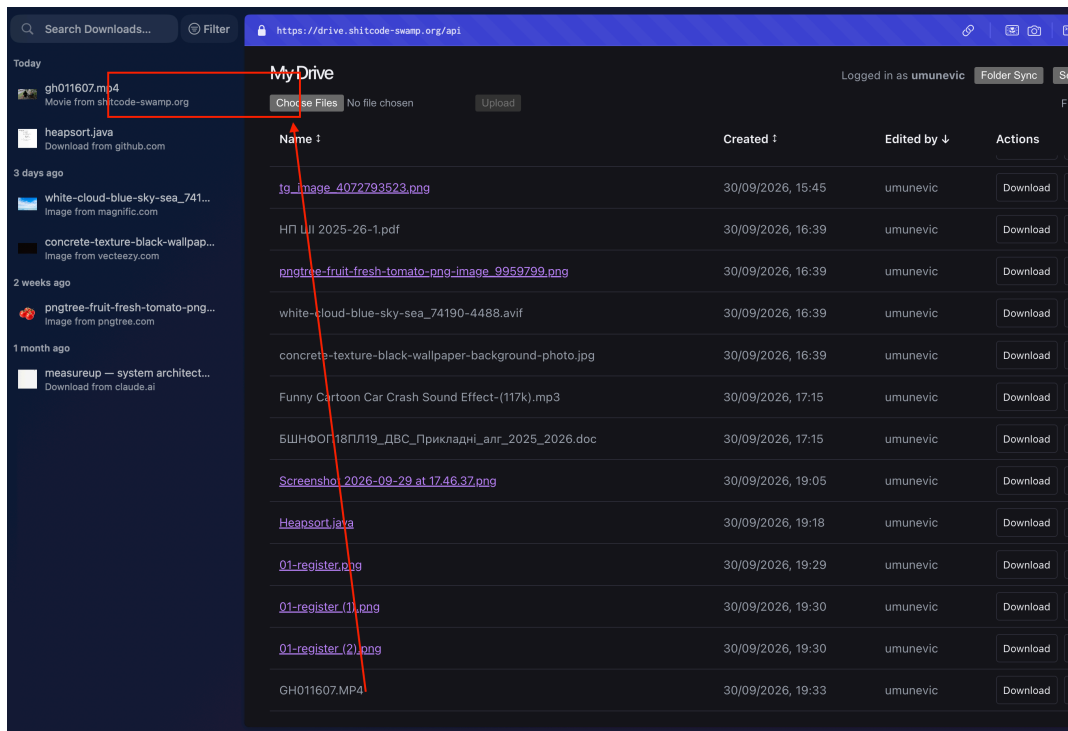


Рис. 11: Завантаження файлу

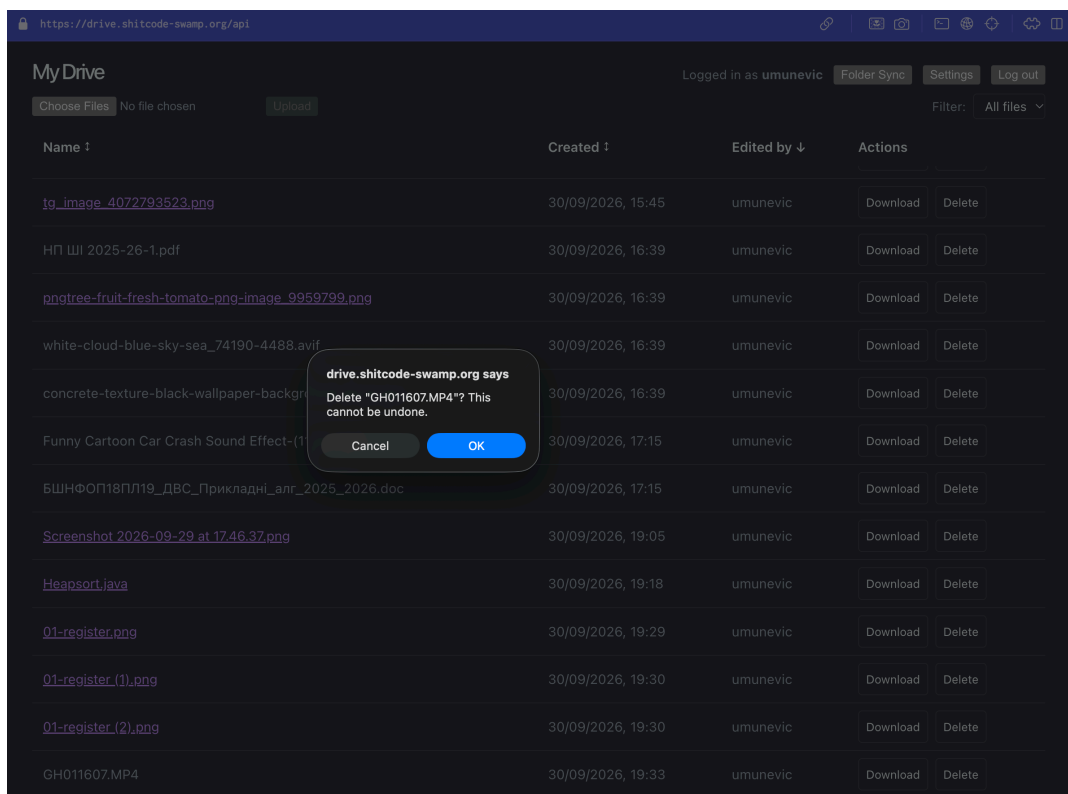


Рис. 12: Видалення файлу

3.5 Синхронізація локальної теки

Рис. 13 показує панель синхронізації теки з обраною текою та щойно завершеною синхронізацією: назва обраної теки, кнопка “Sync now” і короткий журнал того, що зробила остання синхронізація. Рис. 14 показує ту саму панель, коли синхронізація повідомляє про справжній конфлікт: файл показано разом із його локальним/віддаленим розміром та редактором, а також діями Keep Local / Keep Remote / Keep Both.

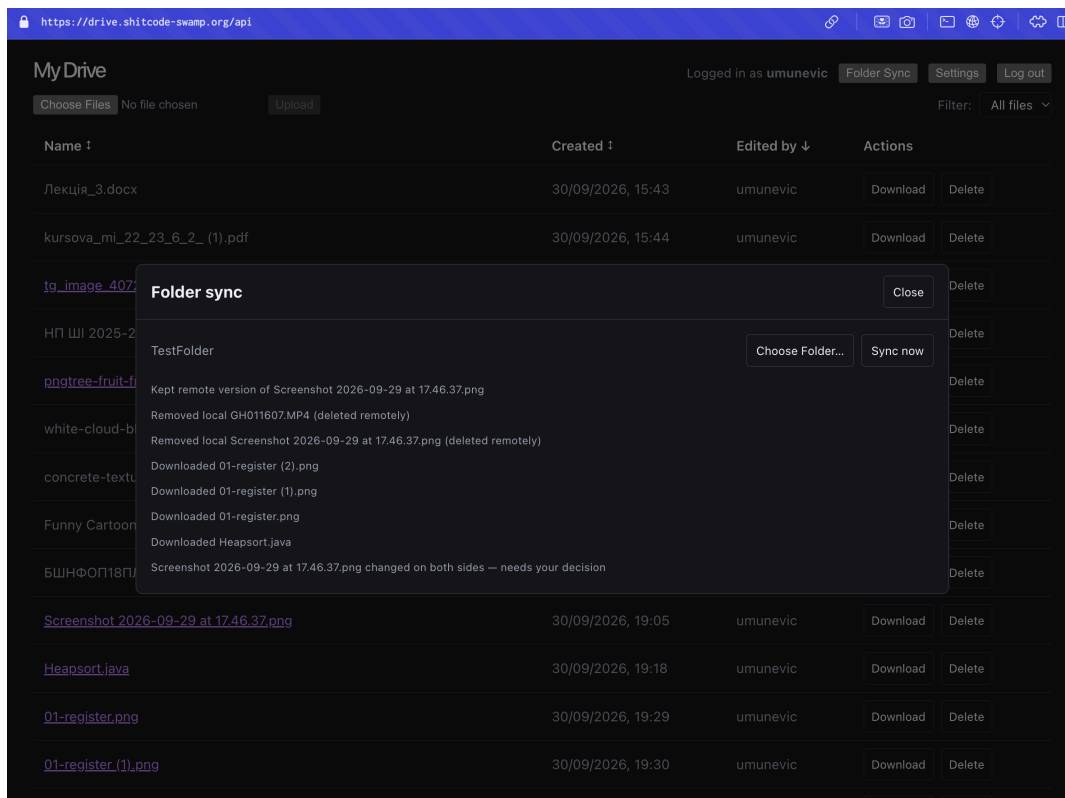


Рис. 13: Панель синхронізації теки після завершеної синхронізації

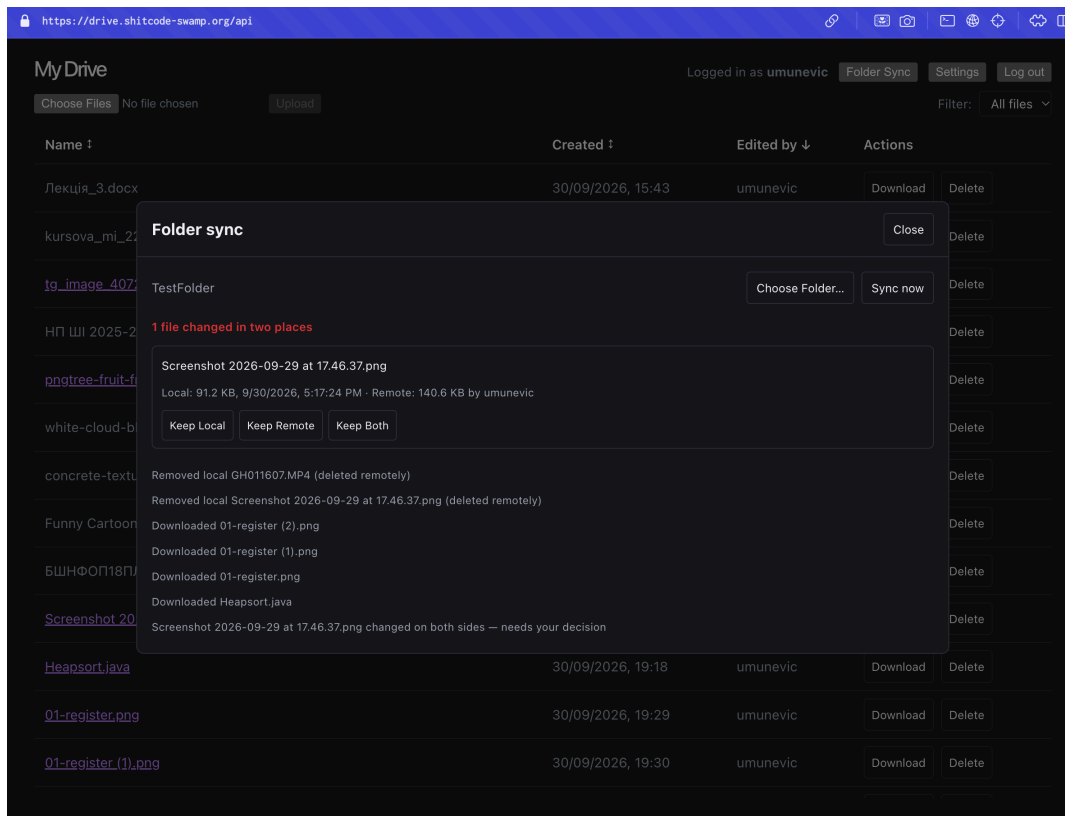


Рис. 14: Панель синхронізації теки з виявленим конфліктом та варіантами його розв'язання

4. Модульне тестування

4.1 Налаштування тестів

Тести веб-клієнта виконуються під Vitest із jsdom, а для тестів компонентів використовується React Testing Library. Мережеві або специфічні для браузера співучасники замінюються, тож набір тестів виконується детерміновано і без реального бекенду, вибору теки в браузері чи IndexedDB:

- `api/client.test.ts` підставляє глобальні `fetch` та `XMLHttpRequest` (останній використовується лише для `chunked`-вивантаження, оскільки `fetch` не має події прогресу вивантаження) через `vi.stubGlobal`;
- `App.test.tsx` підставляє заглушки лише для `listFiles/uploadFileInChunks` з модуля API-клієнта, залишаючи решту експортів (зберігання токенів, хук слухача неавторизованого доступу, ...) реальними, тож гідратація `AuthContext` поводить себе так само, як у продакшені;
- `sync/reconcile.test.ts` та `sync/folderScanner.test.ts` використовують невелику самописну `in-memory` заглушку `FileSystemDirectoryHandle/FileSystemFileHandle` (у `jsdom` немає реального `File System Access API`) та заглушку API-клієнта, що дзеркалить функції, експортовані реальним модулем;

- `sync/storage.ts` написано за протоколом сховища ключ-значення, який можна підставити, саме щоб `sync/storage.test.ts` міг використати `in-memory Map` замість реального `IndexedDB` (також відсутнього в `jsdom`);
- `sync/hash.test.ts` не потребує жодної заглушки – вбудований у `Node crypto.subtle` реалізує справжній `Web Crypto API`, тож тест хешує реальні байти і порівнює результат зі стандартним вектором `sha256("")`.

4.2 Покриття за областями

Таблиця 1 підсумовує набір тестів.

Табл. 1: Тестові файли у `frontend/src/`

Тестовий файл	К-сть	Що покриває
<code>api/client.test.ts</code>	11	Додавання Bearer-токена, політика “одне оновлення – одна повторна спроба” на 401 (обидва шляхи – успіх і провал самого оновлення), декодування відповіді, включно із заголовком <code>X-Has-More</code> , та <code>chunked</code> -шлях вивантаження: розбиття, прогрес по <code>chunk</code> ’ах, переривання при невдачі, повтор після 401 посередині вивантаження й порогове значення для одноразового проти <code>chunked</code> вивантаження
<code>auth/AuthContext.test.tsx</code>	5	Переходи стану входу/реєстрації/виходу, збереження в <code>localStorage</code> під спільними ключами токенів та гідратація вже залогіненого користувача зі сховища при монтуванні без нового мережевого запиту
<code>App.test.tsx</code>	3	Незалежність переліку опцій випадного списку розширень від поточного застосованого фільтра, а також вивантаження методом <code>drag-and-drop</code> , включно з появою та зникненням накладки перетягування
<code>sync/hash.test.ts</code>	2	Кодування SHA-256 у hex, включно зі стандартним тестовим вектором <code>sha256("")</code>
<code>sync/folderScanner.test.ts</code>	3	Сканування теки: обчислення розміру/mtime/sha256, пропуск прихованих файлів і підкаталогів та окреме повідомлення про непрочитаний файл на відміну від відсутнього
<code>sync/storage.test.ts</code>	5	Round-trip стану синхронізації через сховище ключ-значення та поведінка <code>selectDirectory</code> – скидання при іншій теці проти збереження при тій самій

Табл. 1: Тестові файли у frontend/src/

Тестовий файл	К-сть	Що покриває
sync/reconcile.test.ts	14	Обробка дій download/delete_local/conflict алгоритмом узгодження, обидва випадки виключення застарілості в межах одного циклу (щойно завантажений файл, щойно видалений сервером файл), усі три способи розв'язання конфлікту та конвенція іменування копії конфлікту
Разом	43	

Сортування та фільтрація самі застосовуються на сервері (параметри запиту sort/order/extension Go-бекенду, перевірені власним набором тестів бекенду); на боці фронтенду App.test.tsx перевіряє, що зміна фільтра надсилає правильний параметр і що власний перелік опцій випадного списку залишається незалежним від нього, а не повторно тестує логіку сортування/фільтрації бекенду вдруге.

```
Test Files  7 passed (7)
Tests      43 passed (43)
```

5. Висновки

Веб-клієнт покриває повний набір основних прецедентів роботи з робочою областю – перегляд із серверним сортуванням і фільтрацією, попередній перегляд вмісту .java/.png, вивантаження (включно з drag-and-drop), завантаження, видалення – а в браузері на основі Chromium ще й синхронізацію теки за тим самим протоколом diff маніфесту та моделлю конфліктів, які використовує десктоп-клієнт, адаптованими до разової дії за запитом замість фонового спостерігача. Його архітектура дотримується того самого шарування, що й решта проекту: тонкий клієнт api/, що говорить у форматі обміну даними Go-бекенду, React Query/Table/Virtual для керування станом сервера та таблицею файлів, а тека sync/ ізолює File System Access API та IndexedDB за тими самими підставними інтерфейсами для тестування, які Swift-клієнт використовує для власної придатності до тестування. Усі 43 тести набору проходять успішно.